2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW) | 979-8-3315-9994-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/CVPRW67362.2025.00066

A Generative AI Game Jam Case Study from October 2024

Josef Spjut
NVIDIA
jspjut@nvidia.com

Abstract

Generative Artificial Intelligence (GenAI) promises to democratize many creative endeavors, from art, to music, to writing. However, video games are an underexplored field for GenAI given the highly multi-modal and interactive nature. In this work, we present a case study game-jam-style game development process (performed over only a few days!) making heavy use of available GenAI tools (as of October 2024) to create a game called Plunderwater: Sunken Treasure, a title selected from among GenAI suggestions. We share our impressions of what worked well and what could use improvement, and hope that this paper will serve as a guide for GenAI tool developers to focus on the highest impact future improvements and as a starting point for future GenAI for game development benchmarks.

1. Introduction

Generative Artificial Intelligence (GenAI) is likely to drastically change the creative process for video games. Developers and artists already use AI assistants to help with their workflows, but with some improvements, it will soon be possible for a single developer to rapidly create a near complete game using these tools. In an effort to test how close existing GenAI tools are to achieving this objective, we present a game-jam-style case study done by one developer (the author of this report) unfamiliar with the Godot game engine, but using various GenAI assistants and tools to generate nearly all of the content.

A *game jam* [2, 3] is a common practice, similar to a hack-a-thon, whereby an individual or team uses a strict time-limited development period to get as much of a game completed as possible in that time. The author of this work is an expert programmer, and has some experience with some of these GenAI tools, so this is representative of a slightly more expert user, but should still reflect the possibilities for general users in the near future. This should be viewed in contrast to other game development roles such as artists or designers who would likely have a very different experience in using these tools.

In this paper, we share the following contributions:

- A description of the development process and the tools used.
- Observations about what worked well and could potentially be deployed today in production.
- Critiques of the weaknesses of the tools, and a call to refine them for future use.

In addition to the above, we provide the game source and AI chat logs through a mix of supplements to this paper and online distribution. Due to the limited time for conducting this experiment, the resulting game is more of a playable demo with no clear objectives for the player. As can be seen by the included game design document, this lack of objectives was not an intentional part of the design, but an artifact of the experimentation process. With more time, the same process could introduce gameplay objectives as described in the game design document, and complete more levels including environments and enemies. It is clear that this approach, which combines the use of AI with human development, can produce a full playable game.

This game jam effort is similar in style to DreamGarden [11], but with much more human interaction than the "single prompt" goal DreamGarden declares. Rosebud AI [4] is another similar tool, but works more like a chatbot directly hooked into the game code with all image and model generation having to come from outside of the tool. There are also various approaches to create games where GenAI queries are a part of the game loop, either through local or cloud-based compute [19]. In the limit, some systems attempt to replace the game engine entirely with only a GenAI model [8, 9, 31], though these experiences tend to be more like a near-game than actual games. However, similar approaches (e.g. Sora [6]) can result in fairly compelling generated videos. In this work, the human took all actions and made all decisions for which tools to use and how to implement all features. The most direct implementations from AI were code that was copy-pasted and images, though human edits for code and images were still required in many cases.

2160-7516/25/$31.00 ©2025 IEEE
DOI 10.1109/CVPRW67362.2025.00066

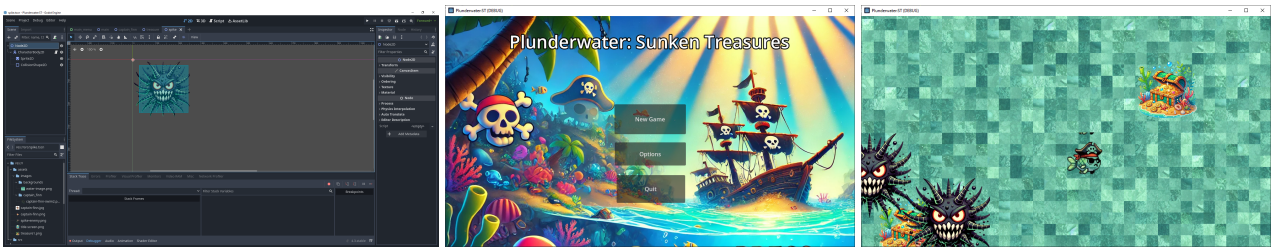

Figure 1. Screenshots of (left) development view of the game in Godot, (center) the main menu, and (right) the gameplay level.

2. Overview

To give an overview of the development process used, we first wanted to figure out how to effectively work on the project from the perspective of a programmer. Next we brainstormed which game to build, created a game design document, and got to work developing as much of the game as possible in the time limit. Since most generative AI tools maintain records of what they generate, we include these as many of these logs as we could obtain after the fact. What follows is a more detailed description of the process and decisions made along the way.

To begin with, we chose to use the Godot game engine [12] since it is a popular open source engine, and wanted to learn how to use it as a secondary goal. Additionally, one of the first tasks that was pre-selected for the AI to help with is to create a usable git-based development flow and distribution system. At the advice of the AI, we used a git repository for the Godot project files and source, but the assets would be uploaded to google drive and an AI-generated (but human modified) script for downloading from google drive was among the first commits to the project. All of the git commits were chosen by the human developer and the commit messages were human generated after certain checkpoints were reached during development.

The following GenAI tools were used in the process of this development:

- Perplexity: General instructions on using Godot
- ChatGPT 4o: Brainstorming (with a brainstorming GPT)
- Gemini: Editing the game design document and concept art
- DALL-E: Image and sprite sheet generation
- Stable Diffusion: Generating the background texture

Since the output from GenAI was not always immediately ready for use, the following non-GenAI tools were used to supplement and clean up the images:

- Image converter [1] to change webp to png.
- Remove.bg [17] to make the background transparent for generated images.

The following chat logs were archived to pdf and are included in the supplement. To see the online chat logs and sources, use these links:

- Brainstorming ChatGPT Session
- Game Design Document
- Perplexity Chat
- Github Repository
- Static sprites and start screen background DALL-E
- Dall-E Image Generation: Only in supplement

2.1. GenAI Tools Used

As described in Section 2, we used a number of different AI assistants for various tasks in this work. At a high level, there are two primary types of assistants used, text chat-bots (Perplexity, ChatGPT, Gemini), and text to image generators (DALL-E [24] and Stable Diffusion [27]). However, in the past year, many of these tools have become multimodal being able to serve both purposes with varying levels of ability and success. For example, ChatGPT directly invokes DALL-E when it identifies a request that should include an image in the response.

In contrast to many text-based chat bots, Perplexity includes a feature that allows it to look up reference material from the web to supplement its responses. This feature is extremely useful, particularly when doing development on the most up-to-date version of a game engine like Godot. Perplexity also allows the use of different underlying large language models (LLM) but we left it on the default setting and did not specify which LLM to use. The other LLMs we used, ChatGPT and Gemini, were selected for more isolated brain storming and ideation that does not benefit as much from live reference material. The choice between ChatGPT and Gemini was somewhat arbitrary, but followed the workflow.

Both ChatGPT and Gemini are able to generate images, via DALL-E in the case of ChatGPT. While there are other types of image generation models, we selected these models mostly out of convenience and proximity to the current workflow. ChatGPT is widely available and many people are familiar with using it. Gemini is integrated into the google docs product suite, and is conveniently available in a side-bar for subscribers. We had a premium Gemini subscription enabled for the duration of this study. While all of the models discussed above are cloud based, we also ran Stable Diffusion through the commonly used Automatic1111 webui on a local machine with an RTX 3090 Ti

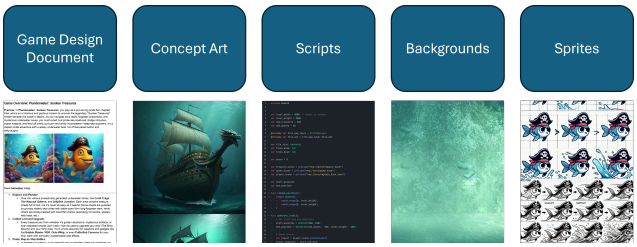

Figure 2. Beginner-level 2D game development output. Traditionally, the assets come from a tutorial and can be optionally created or modified by the developer. When using GenAI these resources come from the AI and the human only needs to execute them.

with 24 GB of VRAM.

2.2. Traditional 2D Game Development

To provide a bit of a contrast to the GenAI-based process to follow, we give a brief overview of more traditional 2D game development as a beginner (see Figure 2). Typically, a user will start by finding a tutorial that will cover a few of the basic setup steps [26]. These tutorials will usually walk the user through some of the game engine interface, and describe steps like how to assign inputs to player movement, how to create a sprite and animation for the player, and something about setting up different scenes and nodes in the scene graph. The scene graph is the data structure that holds all of the objects that are active at the same time in a level. These tutorials either provide all of the sprites and sprite sheets to use, or expect the user to know how to create them by using a drawing program. We did not explicitly use any of these traditional tutorials in this process, though Perplexity used some as a reference in providing instructions.

3. Development Process

The development started with brainstorming, where GPT4o through ChatGPT helped select a game to develop. The initial prompt was:

> I'd like some ideas for a theme for a top down video game. Ideally they'd include some humorous aspects. Perhaps a good pun?

From the list, we selected the title "Plunderwater" (a variation or misreading of "Punderwater" generated by the AI) and provided the follow up prompt:

> I like the last one, but let's just call it Plunderwater and try for a new subtitle. What are some subtitle options?

Inspired by the resulting list of options, the human came up with the following prompt to seed the game design document with an overview of the game:

Figure 3. Concept art from Gemini for the protagonist, Captain Finn, a name chosen by GenAI.

> Okay, let's go with "Plunderwater: Sunken Treasures". Could you give me an overview of the game based on that title. Remember to include some comedy. Highlight the core gameplay loop

At this point, we copied the output from GPT4o into a google doc, where Gemini is integrated to interact directly. Unfortunately, Gemini chats do not save automatically, so the details of that exchange cannot be repeated here. However, Gemini aided in developing various details of the gameplay and levels and the result of those exchanges are reflected in the final game design document linked in Section 2. In addition to the game design, Gemini also generated concept art included in that document including the main character "Captain Finn" (see Figure 3) and the first level "Coral Crags" (see Figure 4).

Equipped with a concrete-enough design to get started, development moved on to using Perplexity to start building the project in Godot Engine (version 3.4) with a focus on getting instructions a human could follow. In general, we asked perplexity for instructions, and followed them or requested more details as needed. At times the follow up detail was needed because of apparent assumed knowledge by the part of the LLM, and at other times the gap appeared to be instructions based on a previous version of the Godot engine. We created a new project and configured for git version control with data coming from google drive, as the earlier selected development preference. Next we created a main menu for the game, and set up some of the initial elements of the game, like the player controlled character.

Figure 4. Concept art generated by Gemini for the first level, Coral Crags.

Similar to many human instructions, Perplexity's instructions often required more familiarity with the interface than the novice human developer had.

Still using perplexity for guidance, we created scenes, characters, and objects for the game. This included a number of scripts in Godot's own GDScript language. Most of the time, it appeared that the code was based on earlier versions of the Godot engine, using a different, but still compatible syntax. On only a couple occasions the code would not compile or run exactly as described, and required manual modification or other follow up to fix. This problem is common when following outdated tutorials, but those tutorials usually specify a version of the tool to use, which is something Perplexity never mentioned. To work around this limitation, we had to use followup requests, or occasional manual edits using the built-in Godot Engine code completion to match the right object and parameter naming and usage, more similar to traditional code development. We never instructed perplexity to write for a particular version of GDScript or Godot Engine though it is likely more effective to do so.

3.1. Characters

The most important character is the player's character, which as discussed previously is called Captain Finn and is a pirate fish. A common concept in 2D games is a *sprite*, which is a 2D image that represents a character or object in the game. Multiple sprites of a single character can describe an animation of that character of some sort, and these

Figure 5. Initial static Captain Finn sprite.

are often encoded into a single image file as a *spritesheet*, commonly one animation per row with a number of frames of animation matching the number of columns.

We seeded a DALL-E session with the concept art and used it to create some sprite images for the main character. Initially during functional prototyping, we used a single static sprite (see Figure 5) for the player character. Later on, in order to get an animation, we tried many prompts to get Dall-E or Stable Diffusion to generate a sprite sheet with many spectacular failures. Eventually, we selected a semi-usable sprite sheet, which can be seen in the original generation, and the edited version in Figure 6.

Additionally, DALL-E generated a treasure sprite and an enemy sprite using a similar graphical style. The title screen was also generated by DALL-E. We used Stable Diffusion to generate a water texture for the background, but the perplexity instructions to add the texture to the game led to slicing it up and displaying the sliced subimages randomly shuffled around. The raw background image texture can be seen in Figure 7, while the result in game can be seen in Figure 1(right). The original intent was to create an image that would naturally tile, something that can easily be done with a stable diffusion plugin, or using photoshop, but the AI instructions changed the development path.

4. Discussion

This experience of trying to complete a game jam as a single human with only a set of AI assistants and tools brought to light certain strengths and weaknesses. The strength and diversity of GenAI tools has been increasing rapidly over the past few years, and we expect it to continue. This discussion reflects only the narrow experience in this paper, while there are many tools and GenAI capabilities that we did not include here. In particular, audio generation tools would be an easy addition, both for sound effects and music [10, 15, 22, 25]. Level generation [28–30] can also be aided by GenAI, but this project did not get far enough to need those models yet. Furthermore, the recent advance-

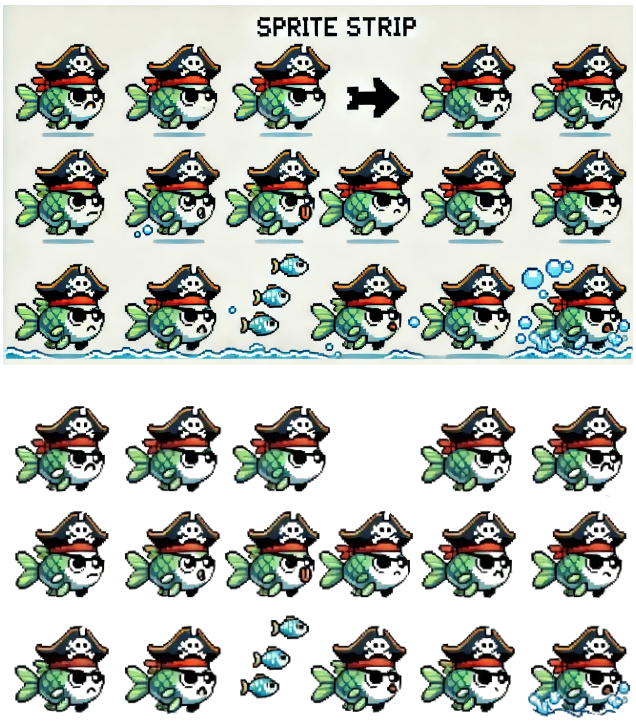

Figure 6. (top) Sprite sheet generated by GenAI and (bottom) edited by a human to make semi-usable by removing the background. Notice how the animation frames are nearly identical instead of showing a swimming fish.

Figure 7. Water image that is sliced and tiled for the level background.

ment of video generation models promises to expand beyond only the image generation used in this effort. While some of the most recent innovations attempt to build end-to-end game models [9, 31], we believe those will primarily see niche application, and the desire for a consistent game that will produce the same range of experiences will benefit from traditional game engine architectures like the one we use in this work.

4.1. Positives

The biggest positive of this approach can be described as what amounts to a "build your own tutorial" for a beginner engine user. Given that the developer had no prior experience with Godot, all of the interactions had to be described by the AI assistant (Perplexity in this case) and then completed by the user. While there are many tutorials on the internet to create similar types of games, and those tutorials likely provide fully working code and examples, this method allowed a custom game with custom assets to be developed. Additionally, the GenAI was easily able to adapt to the level of detail and instruction that the user needed by follow up questions and instructions. Perplexity, with its ability to search for information from the web, was able to provide citations and documentation as needed as well.

Another potential positive, though the legal framework is still in progress [16, 23], is that while traditional tutorials include copyrighted assets that make it difficult to use the end result of a tutorial as the basis for a full product, AI generated assets may be able to avoid copyright and licensing concerns. Of course there may still be moral and ethical reasons to want to create or edit assets in other more manual ways, and the essence of artistic expression desires a human touch in many cases. Still, the ability to have AI complete the components of a game that are not fundamental to the artistic expression is empowering to the individual.

Perhaps future GenAI assistants can be set up to function more explicitly as tutors in an even more effective manner. For example, it would be helpful to have a background of the user as reference for an LLM to know what things to give more or fewer details on. Furthermore, specific reference material for particular tools under instruction, such as the Godot engine, would greatly increase the utility of instructions that are generated.

4.2. Negatives

Even with all the good, there were many limitations of current GenAI models and capabilities. For example, many of the responses giving instructions and code for Godot engine were based on earlier versions of the engine despite being instructed to use the right version (3.4 in this case). This means that much of the Godot script code is using older syntax, and occasionally deprecated types for some game objects. On the image generation side, despite being told repeatedly to use "top down" or "isometric" views for the scenes and objects, all of the generated images ended up being side views instead. Perhaps a fine tuned image

generation or a LoRA [14] for the right perspective would help. Another alternative would be to generate 3D models, and then render the correct perspective, but that is outside the scope of this effort. In general, image generation tools like DALL-E and Stable Diffusion [27] do not support customization. The most common workflow if, for example, the user wants to make the character hold a particular object, is to add that object to the prompt, and generate a bunch of images until one is close enough to what is desired.

Maybe more critically, all of the image generation models we tried were terrible at generating sprite sheets, which are essential to animating the 2D objects. As can be seen in Figure 6 and in the ChatGPT log for the sprite sheet generation, nearly all generated sprite sheets have two major issues. First, the individual animation frames are nearly unchanged, which result in at most very minor animations that do not tend to reflect the requested animation from the prompt. The second major issue is that the placement of the sprites in the sprite sheet do not follow the standard grid layout assumed by most sprite tools, including the Godot engine. These combine to create weird behavior where in this case, sprites from different rows, which are typically used for different animations, had to be used for the purpose of their vertical offset to create the impression of the fish swimming despite the fins of the fish remaining still.

We since did an investigation of other options for direct sprite sheet generation, and found that essentially all currently available tools describe online exhibit the exact same issues described here. There are some quite recent attempts to create animations, which can then be sampled to create the spirtes to use in a sprite sheet. As an example, Zhang et. al [34] generates scalable vector graphics (SVG) animations which could be sampled to create a spritesheets. We leave the full exploration of this space to future work.

4.3. Agentic and Other Systems

Since this game jam, agentic development systems integrated with development environments like Cursor and Windsurf have gained popularity. While we did not compare against them, agentic systems are promising ways to augment or replace many parts of the game development process. In a future experiment, we hope to be able to compare these earlier results to more modern tools.

It is also worth mentioning that machine-learning-centered approaches attempt to fully replace a game engine. For example, DIAMOND [5], GameNGen [32], Genie [7, 21], Oasis [9], and WHAM [18] all create game-style results with a diffusion-style core model. Others use video-based generative models to enable game-like actions or camera motion, for example Promptable Game Models [20], The Matrix [13], GameGen-X [8], and GameFactory [33]. We expect similar models to develop in the near future, some of which may be useful components even when developing content inside of traditional game engines.

5. Conclusion

We have shared one case study of using GenAI tools to create a game-jam-style game over a couple days with a single developer. While a functional game and development flow came out of the experience, there were many limitations. Spritesheets are a particular difficulty for current image generation tools that could use some improvement. However, even with the limitations, GenAI can serve as a meaningful way to get a customized tutorial for an engine like Godot, which can help the person develop skills while getting a meaningfully customized game as a result. We hope that future GenAI advancements will reach beyond the current customized tutorial space and into full-fledge game development assistants in the near future.

6. Acknowledgements

We thank the anonymous reviewers for their feedback that was used to improve this manuscript. Alex Zook and Jonathan Tremblay provided useful discussions about some of the limitations found in the GenAI process and suggested relevant references. Alex Zook, Ruth Rosenholtz, and Joohwan Kim gave useful feedback on earlier drafts.

References

[1] Convertio File Converter — convertio.co. https://convertio.co/. [Accessed 2024-11-04].

[2] Global game jam. https://globalgamejam.org/. [Accessed 24-03-2025].

[3] Game jams — itch.io. https://itch.io/jams. [Accessed 24-03-2025].

[4] Rosebud AI: Build Games at the Speed of Thought. AI Powered Game Development — rosebud.ai. https://www.rosebud.ai/. [Accessed 2024-11-04].

[5] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos J Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37: 58757–58791, 2024.

[6] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024.

[7] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

[8] Haoxuan Che, Xuanhua He, Quande Liu, Cheng Jin, and Hao Chen. Gamegen-x: Interactive open-world game video generation, 2024.

[9] Decart, Julian Quevedo, Quinn McIntyre, Spruce Campbell, and Robert Wachen. Oasis: A universe in a transformer. 2024.

[10] Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music. *arXiv preprint arXiv:2005.00341*, 2020.

[11] Sam Earle, Samyak Parajuli, and Andrzej Banburski-Fahey. Dreamgarden: A designer assistant for growing games from a single prompt. *arXiv preprint arXiv:2410.01791*, 2024.

[12] Godot Engine. Godot Engine - Free and open source 2D and 3D game engine — godotengine.org. https://godotengine.org/. [Accessed 2024-11-04].

[13] Ruili Feng, Han Zhang, Zhantao Yang, Jie Xiao, Zhilei Shu, Zhiheng Liu, Andy Zheng, Yukun Huang, Yu Liu, and Hongyang Zhang. The matrix: Infinite-horizon world generation with real-time moving control. *arXiv preprint arXiv:2412.03568*, 2024.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[15] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M Dai, Matthew D Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer. *arXiv preprint arXiv:1809.04281*, 2018.

[16] Weijie Huang and Xi Chen. Does generative ai copy? rethinking the right to copy under copyright law. *Computer Law & Security Review*, 56:106100, 2025.

[17] Kaleido. Remove Background from Image for Free – remove.bg — remove.bg. https://www.remove.bg/. [Accessed 2024-11-04].

[18] Anssi Kanervisto, Dave Bignell, Linda Yilin Wen, Martin Grayson, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Tabish Rashid, Tim Pearce, Yuhan Cao, et al. World and human action models towards gameplay ideation. *Nature*, 638(8051):656–663, 2025.

[19] Jialu Li, Yuanzhen Li, Neal Wadhwa, Yael Pritch, David E. Jacobs, Michael Rubinstein, Mohit Bansal, and Nataniel Ruiz. Unbounded: A generative infinite game of character life simulation. *arxiv*, 2024.

[20] Willi Menapace, Aliaksandr Siarohin, Stéphane Lathuilière, Panos Achlioptas, Vladislav Golyanik, Sergey Tulyakov, and Elisa Ricci. Promptable game models: Text-guided game simulation via masked diffusion models. *ACM Transactions on Graphics*, 43(2):1–16, 2024.

[21] Jack Parker-Holder, Philip Ball, Jake Bruce, Vibhavari Dasagi, Kristian Holsheimer, Christos Kaplanis, Alexandre Moufarek, Guy Scully, Jeremy Shar, Jimmy Shi, Stephen Spencer, Jessica Yung, Michael Dennis, Sultan Kenjeyev, Shangbang Long, Vlad Mnih, Harris Chan, Maxime Gazeau, Bonnie Li, Fabio Pardo, Luyu Wang, Lei Zhang, Frederic Besse, Tim Harley, Anna Mitenkova, Jane Wang, Jeff Clune, Demis Hassabis, Raia Hadsell, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 2: A large-scale foundation world model. https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/, 2024. Accessed: 2025-03-31.

[22] Damjan Prvulovic, Richard Vogl, and Peter Knees. Restylemusicvae: Enhancing user control of deep generative music models with expert labeled anchors. In *Adjunct Proceedings of the 30th ACM Conference on User Modeling, Adaptation and Personalization*, pages 63–66, 2022.

[23] João Pedro Quintais. Generative ai, copyright and the ai act. *Computer Law & Security Review*, 56:106107, 2025.

[24] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1 (2):3, 2022.

[25] Adam Roberts, Jesse Engel, Colin Raffel, Curtis Hawthorne, and Douglas Eck. A hierarchical latent vector model for learning long-term structure in music. In *International conference on machine learning*, pages 4364–4373. PMLR, 2018.

[26] Rafael Rodríguez. How to do top-down game movement in Godot · GDQuest — gdquest.com. https://www.gdquest.com/tutorial/godot/2d/top-down-movement/, 2021. [Accessed 14-11-2024].

[27] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[28] Shyam Sudhakaran, Miguel González-Duque, Matthias Freiberger, Claire Glanois, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

[29] Graham Todd, Sam Earle, Muhammad Umair Nasir, Michael Cerny Green, and Julian Togelius. Level generation through large language models. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*, pages 1–8, 2023.

[30] Ruben Rodriguez Torrado, Ahmed Khalifa, Michael Cerny Green, Niels Justesen, Sebastian Risi, and Julian Togelius. Bootstrapping conditional gans for video game level generation. In *2020 IEEE Conference on Games (CoG)*, pages 41–48. IEEE, 2020.

[31] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. *arXiv preprint arXiv:2408.14837*, 2024.

[32] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *The Thirteenth International Conference on Learning Representations*, 2024.

[33] Jiwen Yu, Yiran Qin, Xintao Wang, Pengfei Wan, Di Zhang, and Xihui Liu. Gamefactory: Creating new games with generative interactive videos. *arXiv preprint arXiv:2501.08325*, 2025.

[34] Sharon Zhang, Jiaju Ma, Jiajun Wu, Daniel Ritchie, and Maneesh Agrawala. Editing motion graphics video via motion vectorization and transformation. *ACM Trans. Graph.*, 2023.